

拓扑排序应用软件个人任务及感想

一 我承担的任务

我在项目中担任组员 D，主要负责数据文件、导入导出与异常处理模块，具体包括 DataParser 解析器、FileManager 文件读写、InputValidator 输入校验，以及异常与边界场景清单、图 1 数据整理、全系课程数据整理、测试报告和用户使用手册等文档工作。我完成了相应代码和作者自测，整理了图 1 和全系课程两份基准数据，并配合 E 的测试工作提供了容错用例依据。

在模块对接过程中，我与 A 核对解析器输出的数据结构，与 B 确认 FileManager 导出接口与 ResultPanel 的联动方式，与 E 对齐解析器容错测试的规则和场景。我的工作主要是准备数据、定义解析规则、说明异常行为，并在发现问题后配合相关成员重新确认接口、调整实现。这些经历让我意识到，数据模块虽然处于底层，但它的输出结构直接影响建图、界面展示和测试验证，接口约定不清楚就会让后续环节反复返工。

二 对算法实践的体会

这次做 DataParser，我最大的体会是“能跑”和“能交付”是两回事。以前写作业，输入合法、输出正确就算完成；这次要面对全角尖括号、行首尾空格、重复边、自环、空行、注释行，还要把解析失败的行号和原因结构化返回。我一开始只想处理标准格式，后来发现任务书明确要求考虑突发情况，才补上了全角转换、重复边警告和错误列表。这让我认识到，工程实践里的“正确”不是理想输入下的正确，而是各种边界条件下都不崩溃、有明确反馈。

FileManager 的导出功能让我对“结果状态”有了新的理解。我最初写 exportTxt 和 exportCsv 时，只想着把序列写进文件，后来才发现任务书要求导出文件必须写明结果是否完整、为什么停止。生成 1000 条和总共只有 1000 条不是同一个结论，文件如果只列序列不写状态，使用者可能把截断结果当成全部。我后来增加了带 isComplete 和 stopReasonText 参数的重载方法，保留了旧

方法用于简单导出。这让我明白，工程交付物不只要能表达结果，还要能表达结果是在什么条件下得到的。

验证方面，我配合 E 提供了 `DataParser` 和 `InputValidator` 的容错用例。以前我觉得测试就是拿几个正常例子跑一遍，这次 E 需要的是能区分“解析器认为合法”和“校验器认为合法”的用例。前期两个模块规则不统一，同一份输入在不同环节得到不同判断，E 的测试就没法建立稳定预期。后来我们把空行、全角符号、重复边、自环等场景的处理方式写清楚，测试才顺利跑通。这让我认识到，测试不是开发完之后的检查，它反过来会暴露设计里没有想清楚的规则。

三 团队合作中的体会

这次合作让我印象最深的就是接口约定的重要性。项目初期接口契约还没完全冻结就直接开工了，我的解析器基于 A 的图结构，B 的界面和导出又基于我的解析结果，但我没先和 A 确认清楚，B 也在我还没确认好的时候就开工了。结果就是我的设计和 A 的接口契约不符，B 的调用方式也和我的实际输出不一致，最后大家不得不重新确认接口、调整实现。这件事让我明白，接口不是“先写再对”，而是“先对再写”，上游一个字段的含义不清楚，下游就要反复改。

和 A 的对接主要集中在解析结果的数据结构上。我一开始想让 `DataParser` 返回独立的边列表，io 包不依赖 `model` 包；但 A 的接口契约要求解析器直接通过 `Graph.addEdge` 建图。这个分歧没在接口冻结前确认清楚，导致我写好的解析器要调整，B 和 C 的建图逻辑也跟着改。后来重新对齐了 `Graph` 接口和解析器输出方式，问题才解决。

和 B 的对接主要是 `FileManager` 导出接口与 `ResultPanel` 的联动。我写 `FileManager` 时应该先和 B 确认结果数据结构、分隔符、是否带序号、完整性信息怎么传、导出当前页还是全部，但我没先确认就开始写，后面发现问题又返工。

和 E 的对接让我体会到规则一致性的重要性。前期 `DataParser` 和 `InputValidator` 规则不统一，空行、全角符号、重复边这些场景两个模块判断不一致，E 测试时经常对不上，测试没法正常推进。后来我重新梳理了场景清单，把每类输入的处理方式写清楚，测试才顺利跑通。

这次合作也让我意识到，发现自己模块的问题之后，不能只等别人来问。解析器接口调整时，B 和 C 的建图逻辑也会受影响，我后来主动把改动原因、影响范围和新的调用方式整理清楚发到群里，比只丢一句“我改了”要省很多沟通。问题本身不可怕，可怕的是别人不知道我改了哪里、为什么改，等她们自己发现时已经又做了一遍。

四 不足与改进

回顾整个过程，我在沟通和接口确认方面有不少需要改进的地方。项目初期我没有仔细阅读接口契约就开工，导致解析器的设计与 A 的约定不符，后续 B、C、E 的工作也受到影响。我习惯先按自己的想法把功能写出来，再考虑别人怎么调用，但这次项目让我认识到，多模块协作时，“先对接口再写实现”比“先写实现再对接口”更重要。以后我会在动手前先确认接口契约，尤其是数据结构、调用顺序和字段含义，避免因理解偏差导致返工。

在文档和规则统一方面，我也有做得不够好的地方。`DataParser` 和 `InputValidator` 的规则没有一开始就对齐，导致 E 的测试无法正常推进。异常与边界场景清单虽然整理了，但部分场景的处理方案在代码中落地得不够及时。以后我会更早地把规则文档和代码实现对齐，并在接口冻结前主动和上下游成员确认，减少联调时的反复。

在测试和时间安排上，我前期对导出功能的完整性标记考虑不足，和 B 的对接也偏晚。后续我会更早地整理高频场景，尤其是解析、导出和异常处理相关的边界情况，并在接口变更后及时同步相关成员，避免因信息不同步导致重复劳动。

五 小结

这次实践让我认识到，数据模块虽然是底层，但解析结果的结构、导出文件的状态说明、解析与校验的规则一致性，都会直接影响建图、界面展示和测试验证。我以前觉得解析和文件读写只是辅助功能，做完才发现它们的接口和规则必须稳定明确，否则问题会一路传到别人那里。过程中因为接口没提前对齐，我走了不少弯路，今后我会更主动地在动手前确认接口、对齐规则，避免返工。